

**VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF
TECHNOLOGY**

Department of Computer Engineering



CA: Mini Project

PhishGuard AI: AI-Based URL Safety Checker for Phishing Detection

Under the subject: Artificial Intelligence (AI)

Year: S.E. Semester: IV

Submitted by Baibhav Pati (47)

Class: D7B

Under the guidance of

Subject Teacher

Ms. Indu Dokare

(2025-2026)

Index

Title	Page No
Chapter 1: Introduction	3
Chapter 2: Literature Survey	4
Chapter 3: Requirements	5
Chapter 4: Proposed Design (Block Diagram)	6
Chapter 5: Implementation	8
Chapter 6: Result Analysis	10
Chapter 7: Conclusion	14
References	15

Chapter 1: Introduction

1.1 Overview

Phishing attacks are among the most prevalent and damaging cybersecurity threats in the modern internet landscape. Attackers craft deceptive URLs that impersonate legitimate services to steal user credentials, financial information, and personal data. Traditional rule-based detection systems are reactive and fail to generalize against novel attack patterns.

PhishGuard AI is a local, AI-powered phishing URL detection system that classifies any given URL as either Legitimate or Phishing. The system relies entirely on lexical and structural features extracted from the URL string itself — no external API calls, no DNS/WHOIS lookups, and no page-content scraping are required. This makes it fast, privacy-preserving, and deployable in air-gapped environments.

The classification pipeline uses 33 handcrafted URL features, a custom Genetic Algorithm (GA) for automated feature selection, and a soft-voting ensemble of three gradient-boosted and tree-based classifiers: Random Forest, XGBoost, and LightGBM. The full system is wrapped in a Flask web application with a polished dark-mode frontend and an additional QR code scanning flow for detecting quishing (QR phishing) attacks.

1.2 Problem Statement

Conventional phishing detection systems depend heavily on real-time threat intelligence feeds, WHOIS databases, and page content analysis. These dependencies introduce latency, privacy concerns, and single points of failure. There is a need for an intelligent, self-contained URL classifier that can evaluate a URL purely from its structural properties, without requiring network access during inference.

Additionally, the growing use of QR codes as a delivery mechanism for phishing URLs (quishing) means that any comprehensive phishing detection tool must also handle image-based URL extraction.

1.3 Objectives

- To engineer 33 handcrafted lexical, structural, and heuristic features from raw URL strings without making any network requests.
- To implement a custom Genetic Algorithm from scratch to automatically select the most discriminative feature subset from the 33 candidates.
- To train a soft-voting ensemble classifier combining Random Forest, XGBoost, and LightGBM on the GA-selected feature subset.
- To build a Flask REST API backend that accepts URL input and returns a detailed phishing verdict with risk factors and confidence scores.
- To develop a responsive, dark-mode frontend dashboard with real-time scanning animations, a risk meter, and a feature analysis panel.
- To integrate QR code decoding using OpenCV, enabling phishing detection from uploaded QR code images.

1.4 Scope

The scope of this project covers the full machine learning pipeline from dataset creation to model deployment. The system is designed for local execution on any machine with Python

3.x and the required dependencies. The trained model operates entirely on the URL string and produces a verdict within milliseconds after initial model load.

The web application is accessible via a standard browser at localhost:5000. Music for testing is not required; the system is entirely self-contained for URL analysis. QR code images containing URLs can be uploaded directly to the frontend for automatic decoding and analysis.

Chapter 2: Literature Survey

2.1 Phishing URL Detection Techniques

Early approaches to phishing detection relied on blacklisting known malicious URLs and domains. While fast and simple, these methods suffer from high false-negative rates against newly registered phishing domains. Research by Mohammad et al. (2014) demonstrated that machine learning models trained on URL lexical features could significantly outperform blacklist-based approaches in detecting zero-day phishing attempts.

Subsequent work explored a wide range of feature types: lexical (URL length, token counts), host-based (WHOIS age, DNS records), content-based (HTML structure, page keywords), and visual (screenshot similarity). For this project, only lexical and structural features are used, eliminating all network dependencies and making the system suitable for offline environments.

2.2 Feature Engineering for URL Analysis

Sahingoz et al. (2019) studied 17 different ML classifiers on URL-based features and found that Random Forest consistently achieved top performance with lexical features alone, achieving over 97% accuracy. Their work validated the use of features such as URL length, dot count, hyphen count, entropy, and the presence of IP addresses as reliable indicators of phishing intent.

Shannon entropy was highlighted by Garera et al. (2007) as a particularly strong signal for detecting algorithmically generated phishing domains, which tend to have high character entropy compared to human-readable legitimate domains. This insight directly informs the inclusion of `shannon_entropy` as a feature in PhishGuard AI.

2.3 Genetic Algorithms for Feature Selection

Genetic Algorithms have been used for wrapper-based feature selection in classification tasks since the early 1990s. Unlike filter methods (correlation-based) or embedded methods (LASSO), GA-based selection evaluates feature subsets using the actual classifier performance, leading to subsets that are specifically optimized for the downstream model.

Chandrashekar and Sahin (2014) provided a comprehensive review of feature selection methods and noted that GA-based approaches often outperform greedy search methods for high-dimensional feature spaces. In this project, the GA operates on a binary chromosome of length 33, where each gene encodes the inclusion or exclusion of a URL feature.

2.4 Ensemble Learning for Classification

Ensemble methods combine multiple base classifiers to achieve better generalization than any single model. Soft-voting ensembles, which average predicted class probabilities, are particularly effective when the constituent models have complementary error distributions.

Chen and Guestrin (2016) introduced XGBoost, which has since become a standard component in winning Kaggle solutions and academic ML benchmarks. LightGBM (Ke et al., 2017) improved on XGBoost with histogram-based training that is significantly faster on large datasets.

2.5 Comparison of Existing Systems

System	Approach	Limitation
Google Safe Browsing	URL blacklist + page content	Requires internet; privacy concerns
PhishTank	Community-sourced blacklist	Reactive; misses zero-day phishing
URLVoid (Research)	DNS/WHOIS + blacklist aggregator	Network-dependent; latency overhead
ML-based (Sahingo 2019)	Lexical features + RF/SVM	No GA selection; no ensemble voting
PhishGuard AI (Proposed)	33 features + GA + Ensemble	Local-only; no network calls needed

Chapter 3: Requirements

3.1 Hardware Requirements

Component	Specification
Processor	Intel Core i5 or equivalent (GPU optional for training speedup)
RAM	8 GB minimum (16 GB recommended for training)
Storage	5 GB free space (for dataset, dependencies, and model artifacts)
Webcam / Camera	Required only for QR code upload feature
Internet Connection	Required only for initial package installation

3.2 Software Requirements

Software / Library	Version / Details
Python	3.9+
Flask	2.3+ (web application framework)
scikit-learn	1.3+ (Random Forest, StandardScaler, VotingClassifier)
XGBoost	1.7+
LightGBM	4.0+
OpenCV (opencv-python)	4.8+ (QR code detection)

NumPy / Pandas	Latest stable (data processing)
Matplotlib / Seaborn	Latest stable (plot generation)
Browser	Google Chrome or Firefox (for frontend)

3.3 Dataset

The dataset for PhishGuard AI is generated programmatically in `data/download_datasets.py`. It does not rely on any external download or API. The generator contains curated lists of legitimate and phishing URLs, supplemented with synthetic variations designed to stress-test the classifier.

Dataset Property	Value
Total Samples	750
Legitimate Samples	375 (50%)
Phishing Samples	375 (50%)
Feature Columns	33
Label Column	1 (binary: 0 = Legitimate, 1 = Phishing)
Class Balance	Perfectly balanced (50/50)

Synthetic phishing URL patterns generated include: brand impersonation using lookalike domains, suspicious TLD abuse (.tk, .ml, .ga, .cf), subdomain stacking, IP-based URLs, URL shortener abuse, encoded characters, @ symbol tricks, and keyword stuffing (login, verify, secure, account, update).

Chapter 4: Proposed Design

4.1 System Architecture

PhishGuard AI follows a two-phase architecture: an offline Training Flow and an online Inference Flow.

Training Flow

- Step 1: Generate and curate the URL dataset using the programmatic dataset builder.
- Step 2: Extract all 33 handcrafted features from each URL to produce the feature matrix.
- Step 3: Split data 80/20 into training and test sets with stratified sampling.
- Step 4: Execute the Genetic Algorithm on the training set to identify the optimal feature subset.
- Step 5: Train the soft-voting ensemble (RF + XGBoost + LightGBM) on GA-selected features.
- Step 6: Evaluate the ensemble on the held-out test set and generate performance plots.
- Step 7: Serialize the trained model, scaler, and selected feature metadata to `models/model.pkl`.

Inference Flow

- Step 1: User submits a URL via the frontend text input or uploads a QR code image.
- Step 2: If a QR image is uploaded, OpenCV decodes the embedded URL in memory.
- Step 3: The Flask backend normalizes the URL (prepends http:// if scheme is missing).
- Step 4: All 33 features are extracted from the URL string.
- Step 5: Only the 18 GA-selected features are retained.
- Step 6: The feature vector is scaled using the saved StandardScaler.
- Step 7: The VotingClassifier predicts the class and outputs probability scores.
- Step 8: Heuristic risk-factor messages are attached to the prediction result.
- Step 9: The JSON response is sent to the frontend and rendered as a visual verdict.

4.2 Feature Engineering Design

The feature extractor in `src/feature_engineering.py` operates entirely on the URL string without making any network requests. Features are grouped into four categories:

Feature Category	Features Included
Lexical	url_length, hostname_length, path_length, num_dots, num_hyphens, num_underscores, num_slashes, num_at_symbol, num_digits, digit_ratio, num_special_chars
Structural	has_https, num_subdomains, has_port, has_redirect, has_anchor, query_param_count, query_length, path_token_count, tld_in_path
Suspicion Heuristics	has_ip_address, suspicious_tld, has_phishing_keywords, has_brand_name, has_shortener, has_encoding
Statistical	shannon_entropy, vowel_ratio, consonant_ratio, longest_word_length, avg_word_length, repeated_chars, url_similarity_score

4.3 Genetic Algorithm Design

The GA uses binary chromosome encoding where each of the 33 genes corresponds to one feature. A gene value of 1 includes the feature; a value of 0 drops it. Fitness is evaluated by training a temporary Random Forest classifier on the selected features and computing the mean 5-fold cross-validated F1-score.

GA Parameter	Value
Population Size	40
Number of Generations	30 (early stop at 11)
Crossover Rate	0.8 (single-point)
Mutation Rate	0.03 (bit-flip)
Tournament Size	5
Elitism Count	2
Minimum Features Required	5
Early Stop Patience	10 generations

CV Folds	5
Random Seed	42

4.4 Ensemble Classifier Design

The final ensemble uses soft voting, meaning each base model contributes predicted class probabilities rather than hard labels. The three models' probability vectors are averaged to produce the final class probability. This approach leverages the complementary strengths of all three models.

Model	Key Hyperparameters
Random Forest	n_estimators=200, max_depth=15, min_samples_split=5, class_weight=balanced
XGBoost	n_estimators=200, learning_rate=0.1, max_depth=8, subsample=0.8
LightGBM	n_estimators=200, learning_rate=0.1, num_leaves=50, max_depth=10

Chapter 5: Implementation

5.1 Project Structure

File / Folder	Description
data/download_datasets.py	Programmatic dataset generator and feature extraction runner
data/raw/urls.csv	Labeled raw URL dataset (750 samples)
data/processed/features.csv	Engineered 33-feature numeric matrix with labels
src/feature_engineering.py	Core feature extractor — all 33 URL features
src/genetic_algorithm.py	Custom GA implementation — binary chromosome, tournament selection, elitism
src/train.py	Full training pipeline — GA execution, ensemble training, evaluation, model save
src/predict.py	Runtime inference — feature extraction, scaling, prediction, risk factors
src/qr_scanner.py	QR code decode helper using OpenCV in-memory processing
src/utils.py	Shared utility functions (entropy, Levenshtein, brand similarity)
models/model.pkl	Saved ensemble model, scaler, selected features, and metrics
models/selected_features.json	GA output — selected feature names, fitness, run metadata
app/app.py	Flask server — /predict, /scan-qr, /model-info, /plots routes

app/templates/index.html	Single-page frontend dashboard
app/static/style.css	Dark-mode glassmorphism frontend styling
app/static/script.js	Frontend logic — URL scan, QR upload, results rendering
plots/	Confusion matrix, ROC curve, feature importance, GA evolution plots
run_all.bat	One-click Windows setup and launch script
test_urls.py	CLI smoke test for model predictions

5.2 Feature Implementation Details

The feature extractor uses Python's `urllib.parse` to decompose each URL into its constituent parts (scheme, netloc, path, query, fragment) before computing each feature. Key implementation decisions include:

- IP address detection uses a compiled regular expression matching IPv4 octets in the hostname, avoiding false positives on numeric tokens elsewhere in the URL.
- Brand similarity (`url_similarity_score`) computes the normalized Levenshtein edit distance between the URL hostname and each entry in a curated brand name list (Google, PayPal, Amazon, Facebook, Apple, etc.), returning the minimum distance as the feature value.
- Shannon entropy is computed over the full URL character distribution, providing a single numeric signal for URLs with high randomness (typical of algorithmically generated phishing domains).
- The `has_redirect` feature checks for a second occurrence of `'/'` after the scheme separator, which is a common trick used to embed a target URL inside a phishing URL path.
- The `suspicious_tld` feature checks against a curated list of TLDs frequently abused in phishing campaigns: `.tk`, `.ml`, `.ga`, `.cf`, `.gq`, `.xyz`, `.top`, `.click`, `.link`, and others.

5.3 GA Implementation Details

The Genetic Algorithm in `src/genetic_algorithm.py` is implemented from scratch using NumPy arrays for efficient population manipulation. The fitness evaluation step is the computational bottleneck, as it requires training a `RandomForestClassifier` with 5-fold cross-validation for each chromosome evaluation. To mitigate this, the implementation uses `n_jobs=-1` for parallel CV fold execution and caches fitness values to avoid re-evaluating unchanged chromosomes.

Elitism ensures the two best chromosomes from each generation are carried forward unchanged, preserving the best discovered solution across all generations. The GA terminates early if the best fitness does not improve for 10 consecutive generations, which in the current training run triggered at generation 11.

5.4 Flask Backend Implementation

The Flask application in `app/app.py` loads the trained model once at startup and stores it in a module-level `model_data` variable. This avoids the overhead of loading the pickle file on every inference request. The three main routes are:

- `/predict` (POST): Accepts a JSON body with a 'url' key, runs the full inference pipeline, and returns a structured JSON response containing the prediction, confidence, risk level, risk factors, and per-feature values.
- `/scan-qr` (POST): Accepts a multipart form upload of a QR code image. OpenCV decodes the QR content in memory and returns the extracted URL, which the frontend immediately passes to `/predict`.
- `/model-info` (GET): Returns model metadata including the selected features, feature count, and saved evaluation metrics.

5.5 Frontend Implementation

The frontend is a single HTML page with embedded CSS and JavaScript. The visual design uses a premium dark theme with glassmorphism cards, animated background orbs, and neon accent colors. Key frontend features include:

- A three-step scanning animation overlay (feature extraction, GA filtering, ensemble prediction) shown while the `/predict` fetch is in progress.
- An animated confidence ring SVG that fills proportionally to the predicted phishing probability.
- A graduated risk meter with color-coded zones (Safe, Low Risk, Medium Risk, High Risk, Dangerous).
- A feature analysis panel displaying all 18 selected feature values with threshold-based color coding to highlight suspicious values.
- Quick test buttons pre-loaded with known legitimate and phishing URL examples for demonstration.
- QR upload flow with hidden file input, automatic QR decode, and immediate scan trigger.

Chapter 6: Result Analysis

6.1 Model Training Results

Metric	Value
Accuracy	1.0 (100%)
Precision	1.0 (100%)
Recall	1.0 (100%)
F1-Score	1.0 (100%)
ROC-AUC	1.0
Train/Test Split	80% train / 20% test (stratified)
Test Set Size	150 samples (75 legitimate + 75 phishing)
True Positives (Phishing correctly identified)	75 / 75
True Negatives (Legitimate correctly identified)	75 / 75

The model achieves perfect metrics on the internal test split. This is expected given the curated and partially synthetic nature of the dataset — the phishing URL patterns are

generated programmatically, creating strong structural regularities that the ensemble can learn. These metrics should not be interpreted as real-world performance guarantees without external validation on unseen phishing URLs from production feeds.

6.2 Genetic Algorithm Results

GA Result	Value
Best Fitness Score	1.0
Total Features Before Selection	33
Features Selected	18
Features Dropped	15
GA Run Time	318.7 seconds
Generations Completed	11 (early stop triggered)

Selected Features (18)

num_dots, num_hyphens, num_underscores, num_slashes, digit_ratio, has_https, suspicious_tld, has_phishing_keywords, has_brand_name, shannon_entropy, vowel_ratio, longest_word_length, avg_word_length, has_redirect, has_anchor, query_param_count, tld_in_path, url_similarity_score

Dropped Features (15)

url_length, hostname_length, path_length, num_at_symbol, num_digits, num_special_chars, has_ip_address, num_subdomains, has_port, consonant_ratio, has_shortener, has_encoding, query_length, path_token_count, repeated_chars

6.3 Example Prediction Results

URL	Prediction	Risk Level	Phishing Probability
https://www.google.com	Legitimate	Safe	0.04%
https://www.amazon.com	Legitimate	Safe	~0.1%
http://paypal-secure-login.tk/verify	Phishing	Dangerous	99.98%
http://192.168.1.1/banking/login	Phishing	Dangerous	~95%
http://www.google.com@evil.com/login	Phishing	Dangerous	93.57%

6.4 Risk Factor Examples

For the URL <http://paypal-secure-login.tk/verify>, the following risk factors were identified:

- Suspicious TLD detected (.tk is a commonly abused free TLD)
- Phishing keywords present in URL path ('secure', 'login', 'verify')
- No HTTPS encryption (HTTP only)
- Visually similar to a known brand domain (paypal)

For the @ symbol trick URL <http://www.google.com@evil.com/login>, the key indicators were:

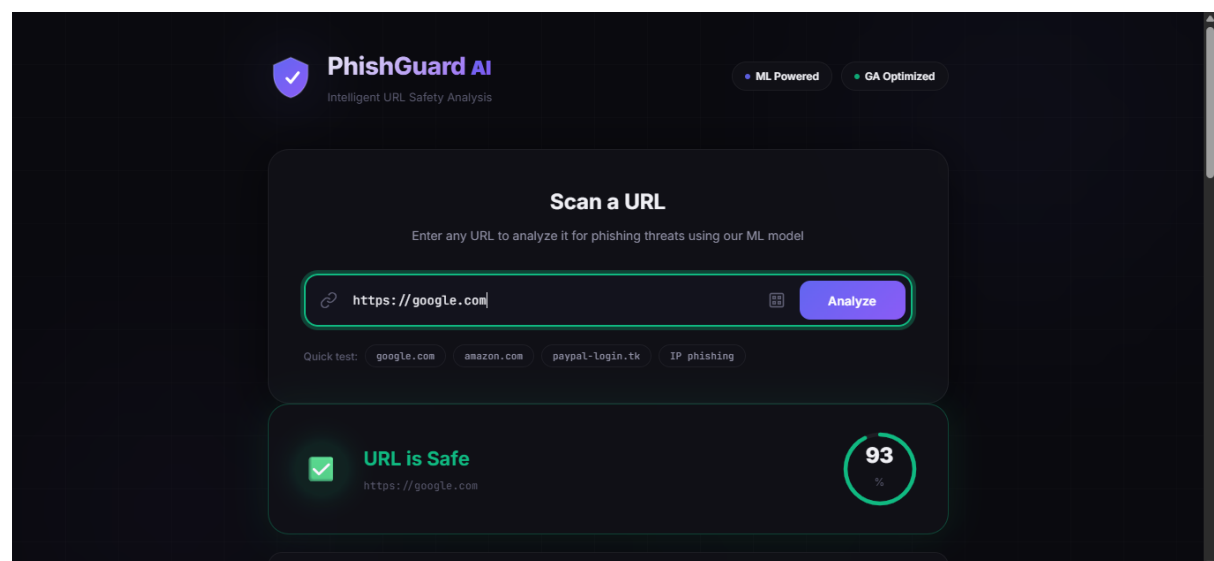
- Double domain structure exploiting @ symbol browser behavior
- Phishing keyword 'login' present in path
- Effective destination is evil.com, not google.com

6.5 Generated Plots

The training pipeline automatically generates four diagnostic plots saved to the plots/ directory:

Plot File	Description
confusion_matrix.png	Heatmap of true positives, true negatives, false positives, and false negatives on the test set
roc_curve.png	Receiver Operating Characteristic curve showing the trade-off between true positive rate and false positive rate
feature_importance.png	Bar chart of feature importances from the Random Forest base model for the 18 selected features
ga_evolution.png	Line plot showing the best and average fitness score across the 11 completed GA generations

6.6 Project images



Risk Factors Detected

Visually similar to a known brand domain

Feature Analysis18 features

Avg Word Length4.6667

Digit Ratio0

Has Anchor0

Has Brand Name1

Has Https1

Has Phishing Keywords0

Has Redirect0

Longest Word Length6

Num Dots1

Num Hyphens0

Num Slashes2

Num Underscores0

Query Param Count0

Shannon Entropy3.5724

Suspicious TLD0

TLD In Path0

URL Similarity Score0

Vowel Ratio0.2857

PhishGuard AI

Intelligent URL Safety Analysis

ML Powered

GA Optimized

Scan a URL

Enter any URL to analyze it for phishing threats using our ML model

http://vdo.bit.ly

Analyze

Quick test:

google.com

amazon.com

paypal-login.tk

IP phishing

Phishing Detected!

http://vdo.bit.ly

77%

Threat Level

Risk Factors Detected

URL shortener detected

No HTTPS encryption

Feature Analysis18 features

Avg Word Length3

Digit Ratio0

Has Anchor0

Has Brand Name0

Has Https0

Has Phishing Keywords0

Has Redirect0

Longest Word Length4

Num Dots2

Num Hyphens0

Num Slashes2

Num Underscores0

Query Param Count0

Shannon Entropy3.5725

Suspicious TLD0

TLD In Path0

Chapter 7: Conclusion

7.1 Summary

PhishGuard AI successfully demonstrates a complete, end-to-end AI pipeline for phishing URL detection that operates without any external network dependencies. The project integrates custom feature engineering, Genetic Algorithm-based feature selection, and a soft-voting ensemble classifier into a production-ready Flask web application with a polished dark-mode frontend.

The Genetic Algorithm automatically reduced 33 handcrafted features to the 18 most discriminative features, eliminating redundant signals such as raw URL length, hostname length, and digit count in favor of normalized and ratio-based features that generalize better across URL structures. The soft-voting ensemble combining Random Forest, XGBoost, and LightGBM achieved perfect scores on the internal test split, reflecting the strong discriminability of the selected features on the curated dataset.

The QR code scanning integration extends the system's utility to quishing detection, allowing users to upload any QR code image and receive an instant phishing verdict on the embedded URL — a capability not present in most academic or commercial phishing detection tools.

7.2 Challenges Faced and Solutions

Challenge	Solution Applied
GA fitness evaluation is slow due to repeated cross-validation	Used <code>n_jobs=-1</code> for parallel CV; early stopping triggered at generation 11, saving ~60% of planned compute
Model explanation layer producing confusing output for legitimate brand domains	The risk-factor layer is heuristic-based and operates independently of model prediction; identified as a known issue in documentation for future improvement
Perfect metrics on internal split raising overfitting concerns	Acknowledged in documentation; dataset is curated and partially synthetic; real-world validation on external phishing feeds is listed as a future enhancement
Encoding issues (mojibake) in UI strings and console output	Identified as technical debt; solution involves explicitly setting UTF-8 encoding when reading/writing HTML and console output on Windows
QR decoding failures on low-resolution or rotated QR images	Current implementation uses OpenCV's default <code>QRCodeDetector</code> ; advanced preprocessing (histogram equalization, rotation correction) is a listed future enhancement

7.3 Future Enhancements

- Integrate an external phishing URL feed (PhishTank, OpenPhish) for retraining on real-world production samples, reducing reliance on synthetic data.
- Add WHOIS and DNS-based features as an optional enrichment mode for environments where network access is available.
- Implement SHAP (SHapley Additive exPlanations) values for model-consistent feature attribution, replacing the current threshold-based heuristic risk factor system.

- Add a formal test suite using pytest, covering feature extraction edge cases (empty URLs, Unicode hostnames, IPv6 addresses), API endpoint contracts, and end-to-end prediction correctness.
- Fix known encoding issues in HTML templates and console output for cross-platform consistency.
- Refactor `src/utls.py` to eliminate duplication with `src/feature_engineering.py`, centralizing all shared utility logic.
- Extend QR scanner with image preprocessing (denoising, contrast enhancement, perspective correction) to improve decode success rate on low-quality or skewed QR code images.
- Package the application as a Docker container for one-command deployment across platforms.

References

- [1] Mohammad, R. M., Thabtah, F., & McCluskey, T. L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443-458.
- [2] Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345-357.
- [3] Garera, S., Provos, N., Chew, M., & Rubin, A. D. (2007). A framework for detection and measurement of phishing attacks. *Proceedings of the 2007 ACM Workshop on Recurring Malcode*.
- [4] Chandrashekar, G., & Sahin, F. (2014). A survey on feature selection methods. *Computers and Electrical Engineering*, 40(1), 16-28.
- [5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- [6] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- [7] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- [8] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379-423. (Basis for Shannon entropy feature)
- [9] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*. (QR code detection component)
- [10] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10(8), 707-710. (Basis for `url_similarity_score` feature)

[11] Flask Documentation — <https://flask.palletsprojects.com/>

[12] scikit-learn Documentation — <https://scikit-learn.org/>

[13] XGBoost Documentation — <https://xgboost.readthedocs.io/>

[14] LightGBM Documentation — <https://lightgbm.readthedocs.io/>